CSE470 · GROUP INFINITY · SPRINT 0

Ayojon Database Report

The design and implementation of the database behind all 20 Ayojon features, how it was verified, and the updated ER diagram.

29	41	18	16
tables	foreign keys	views	triggers

Prepared by	Tasnia Islam (24241225)
Team	Mohammed Sabbir Ahmed · Tasneem Akram Nawshin · Nuzhat Tabassum Nora · Tasnia Islam
Branch	`feature/tasnia-database` on github.com/sabbirx06/ayojon
Date	29 October 2026

1 Summary

I built the complete working database for Ayojon on its own branch. It extends the team's normalized schema from Sprint 0 so that every one of the 20 features has the tables, rules and queries it needs. The database is created automatically when the Spring Boot app starts, so nobody needs to run SQL by hand.

- **29 tables** with 190 columns and 41 foreign keys, in third normal form.
- **16 triggers** and **21 functions** that enforce the business rules inside PostgreSQL, so they hold whichever code writes the data.
- **18 views** and query functions that answer each feature's read question: search, nearby, trending, recommendations, leaderboard, dashboard and more.
- **Java data-access layer:** 28 JPA entities, 9 enums, 28 Spring Data repositories and 17 typed projections.
- **Demo data** of 16 real Dhaka events and 9 demo accounts that goes through the real rules.
- **Supabase security** so the public REST API cannot read our tables.
- **Tests:** 49 SQL rule checks and 6 Java tests, all passing on a database built from empty.

Technology stack

LAYER	CHOICE	NOTES
Database	PostgreSQL on Supabase	Written for PostgreSQL 15+; tested on 17.5
Migrations	Flyway 12.4	Version managed by Spring Boot; runs on app start
Backend	Spring Boot 4.1.2, Java 21	As set up in the project skeleton
Data access	Spring Data JPA, Hibernate 7.4, Lombok	`ddl-auto=validate`: Hibernate checks tables, never changes them

2 Migrations

The schema lives in versioned SQL files under `src/main/resources/db/migration`. Flyway applies them in order and records them in `flyway_schema_history`. An applied file is never edited; changes go into a new file such as `V6__…sql`.

FILE	CONTENTS
`V1__core_schema.sql`	All 29 tables, primary, foreign and unique keys, CHECK constraints and indexes, including the generated search vector with a GIN index.
`V2__functions_and_triggers.sql`	Business-rule triggers and helpers such as `event_is_verified()` and `haversine_km()`.
`V3__feature_views_and_queries.sql`	18 read views and the query functions `search_events`, `events_nearby`, `recommend_events` and `find_overlapping_events`.
`V4__reference_data.sql`	11 categories, 14 Dhaka areas, 20 tags, 7 dispute reasons, 5 badges.
`V5__supabase_security.sql`	Row Level Security on every table and API roles' access revoked.

FILE	CONTENTS
`db/seed/R__demo_data.sql`	Repeatable demo data, loaded once per database. Can be switched off for production.

Tables by area

TABLE	PURPOSE	FEATURES	DEMO ROWS
Accounts & lookups			
`users`	All accounts with role USER / ORGANIZER / ADMIN. `points` is a trigger-maintained total.	F3, all	9
`event_category`	Primary category of an event.	F5, F16	11
`tag`	Free-form tags.	F16, F19	20
`area`	Dhaka neighbourhoods with centre points.	F5, F20	14
`venue`	Places with address and GPS coordinates.	F1, F2, F6	12
`dispute_reason`	Lookup of report reasons (SPAM, COUNTERFEIT_LISTING, …).	F14	7
`badge`	Milestone badges and the points needed.	F3	5
Events			
`event`	Event listing with lifecycle status, times, links, check-in points and a generated search vector.	F5, F6, F16–F18	16
`event_tag`	Event ↔ tag link.	F16, F19	41
`event_media`	Posters, images and videos (one poster per event).	F16	19
`ticket_tier`	Ticket price tiers; a free event has a tier priced 0.	F5, F6	21
Moderation & audit			
`event_submission_review`	Admin decisions on submitted events.	F18	14
`event_verification`	Legitimacy checks; the latest decision gives the Verified badge.	F7	7
`event_change_log`	Field-level audit trail of every edit.	F17	17
`dispute_report`	User reports and their resolution.	F14	3
User activity			
`attendance`	Check-ins with points awarded.	F2, F3, F20	9
`review`	Rating 1–5 and short review; only for attendees.	F15	6
`bookmark`	Saved events; drives trending and recommendations.	F8, F9, F19	18
`collection`	Named collections and private wishlists.	F9	3

TABLE	PURPOSE	FEATURES	DEMO ROWS
`collection_item`	Bookmarked events inside a collection.	F9	6
`itinerary`	Weekend plans (up to 7 days).	F10	1
`itinerary_item`	Events in a plan with optional partial slots.	F10, F11, F13	4
`event_follow`	Users following an event.	F12	7
`notification`	In-app notifications, mostly written by triggers.	F12, F18, F3	32
`user_badge`	Badges each user has unlocked.	F3	5
Sponsors & vouchers			
`sponsor`	Sponsor companies.	F4	3
`ad_campaign`	Sponsor ad campaigns that generate vouchers.	F4	2
`voucher`	Promo codes with discount, stock, validity and points threshold.	F4	3
`voucher_claim`	Vouchers claimed and redeemed by users.	F4	4

3 Changes from the draft schema

The Sprint 0 draft (`normalized_schema.sql`) was the starting point. Checking it against each feature showed these gaps, which the final schema fills. The draft files are kept in `docs/database/archive` .

CHANGE	REASON
Keys and times	`SERIAL` → `BIGINT` identity columns; all timestamps `TIMESTAMPTZ` .
Event lifecycle	Status vocabulary for the approval flow: DRAFT, PENDING_REVIEW, REVISION_REQUESTED, PUBLISHED, REJECTED, CANCELLED. COMPLETED dropped because it is derivable from `end_time` .
New: event_submission_review	Separates the F18 approval decision from the F7 legitimacy check, which the draft mixed together.
New: bookmark	One "saved" fact per user and event, so trending (F8) and recommendations (F19) don't double-count an event saved into two collections.
collection_event → collection_item	Composite foreign keys tie each item to the owner's own bookmark.
New: tag, event_tag	Covers the "category tags" of F16 and feeds recommendations.
New: ad_campaign	F4 says vouchers are generated from sponsor ad campaigns; vouchers gained stock, points threshold and validity window.
event_change_log	Now field-level (field, old value, new value, change type) and written automatically.
review	Must reference an attendance row (F15 "after attending").
dispute_report	Records who resolved it, the resolution note and the time.

CHANGE	REASON
Removed	`user_badge.attendance_id` and `itinerary_item.sequence_no`; order now comes from times. The `event_verified` view was replaced by `v_event_verification_status`.
Search	Generated `tsvector` column with a GIN index for keyword search (F5).

Normalization

- Venue and area are separate tables, so an event stores only `venue_id` and has no transitive dependency on address or coordinates.
- Facts that can be derived are not stored. Verified status, minimum price, average rating, bookmark counts and the trending rank are computed in views.
- One deliberate cache is `users.points`, the sum of `attendance.points_awarded`. It keeps the leaderboard fast, and a trigger keeps it exactly in sync.
- Lookups (`event_category`, `area`, `tag`, `dispute_reason`, `badge`) replace free-text values.
- Status values are VARCHAR with a CHECK constraint, which maps directly to Java enums.

4 Rules the database enforces

These rules run in PostgreSQL, so they hold no matter which part of the app writes the data. A violation comes back to Spring as an exception with a readable message.

RULE	BEHAVIOUR
Event submission (F18)	Saving an `event_submission_review` moves the event to PUBLISHED, REJECTED or REVISION_REQUESTED and notifies the organizer. Only PENDING_REVIEW events can be reviewed. Rejections need a comment.
Verification (F7)	A VERIFIED decision must cite a `source_url`. The organizer is notified.
Audit trail (F17)	Every update of an event writes one `event_change_log` row per changed field, with the editor taken from `event.updated_by`.
Follower alerts (F12)	Followers are notified when a published event is rescheduled, moves venue or is cancelled. Cancelling requires a reason.
Check-ins (F3)	Only PUBLISHED events accept check-ins, once per user. Points are awarded only at verified events. `users.points` is recomputed and badges are granted and announced.
Reviews (F15)	A composite foreign key to `attendance` means only users who checked in can review, once per event.
Collections (F9)	Adding an event to a collection fills in the owner and bookmarks it. Composite foreign keys keep collection and bookmark owned by the same user.
Itineraries (F10)	Items must be PUBLISHED events inside the plan's dates. A custom slot must lie within the event's own times.
Vouchers (F4)	A claim is checked for active status, validity window, stock (row-locked against races) and the user's points.

RULE	BEHAVIOUR
Disputes (F14)	One open report per user, event and reason. Only admins resolve; resolving stamps the time and notifies the reporter.
Roles (all)	Only ORGANIZER or ADMIN can own events; only ADMIN can moderate, verify or resolve. `updated_at` is maintained automatically.

Triggers

TABLE	TRIGGER	FIRES
`attendance`	`trg_attendance_after_change`	AFTER DELETE, INSERT, UPDATE
`attendance`	`trg_attendance_before_insert`	BEFORE INSERT
`collection`	`trg_collection_updated_at`	BEFORE UPDATE
`collection_item`	`trg_collection_item_before_insert`	BEFORE INSERT
`dispute_report`	`trg_dispute_report_before_update`	BEFORE UPDATE
`event`	`trg_event_after_update`	AFTER UPDATE
`event`	`trg_event_before_write`	BEFORE INSERT, UPDATE
`event`	`trg_event_updated_at`	BEFORE UPDATE
`event_submission_review`	`trg_submission_review_apply`	BEFORE INSERT
`event_verification`	`trg_event_verification_check`	BEFORE INSERT
`event_verification`	`trg_event_verification_notify`	AFTER INSERT
`itinerary`	`trg_itinerary_updated_at`	BEFORE UPDATE
`itinerary_item`	`trg_itinerary_item_check`	BEFORE INSERT, UPDATE
`review`	`trg_review_updated_at`	BEFORE UPDATE
`users`	`trg_users_updated_at`	BEFORE UPDATE
`voucher_claim`	`trg_voucher_claim_before_insert`	BEFORE INSERT

5 Feature coverage

Every feature from the project document maps onto the schema. Each read query is wrapped in a repository method, listed in `docs/database/README.md`.

#	FEATURE	OWNER	TABLES	READ MODEL
F1	Nearby events by GPS radius	Nora	`venue`	events_nearby()
F2	Attendance heat map	Sabbir	`attendance, venue`	v_user_heatmap

#	FEATURE	OWNER	TABLES	READ MODEL
F3	Points, badges, leaderboard	Sabbir	`attendance, badge, user_badge, users.points`	v_leaderboard + triggers
F4	Sponsor discount vouchers	Sabbir	`sponsor, ad_campaign, voucher, voucher_claim`	v_voucher_availability
F5	Search by keyword, date, area, category, price	Tasnia	`event, venue, area, ticket_tier`	search_events()
F6	Event profile	Tasnia	`event, ticket_tier, event_media`	v_event_listing
F7	Verified trust badge	Nawshin	`event_verification`	v_event_verification_status
F8	Trending by weekly saves	Tasnia	`bookmark`	v_trending_events
F9	Collections & wishlists	Tasnia	`collection, collection_item, bookmark`	repositories
F10	Weekend schedule	Tasnia	`itinerary, itinerary_item`	v_itinerary_schedule
F11	Overlap detection	Nawshin	`event, itinerary_item`	find_overlapping_events(), v_venue_event_overlap, v_itinerary_conflict
F12	Follow events + notifications	Nawshin	`event_follow, notification`	trigger on event update
F13	Calendar export (.ics / CSV)	Nora	`itinerary_item, event, venue`	v_calendar_export
F14	Dispute reports	Nora	`dispute_report, dispute_reason`	v_dispute_queue
F15	Ratings & reviews	Nawshin	`review → attendance`	composite FK
F16	Submit event	Nora	`event, event_media, ticket_tier, event_tag`	repositories
F17	Edit / cancel with public log	Nora	`event_change_log`	v_public_event_change_log
F18	Admin approval & organizer status	Nawshin	`event_submission_review`	v_admin_submission_queue, v_organizer_submission_status
F19	Recommendations from bookmarks	Sabbir	`bookmark, event_tag`	recommend_events()
F20	Attendance dashboard	Sabbir	`attendance`	v_user_attendance_summary, v_user_monthly_attendance, category/area stats

Views and query functions

`v_admin_submission_queue`, `v_calendar_export`, `v_dispute_queue`, `v_event_listing`, `v_event_verification_status`, `v_itinerary_conflict`, `v_itinerary_schedule`,

`v_leaderboard`, `v_organizer_submission_status`, `v_public_event_change_log`, `v_trending_events`, `v_user_area_stats`, `v_user_attendance_summary`, `v_user_category_stats`, `v_user_heatmap`, `v_user_monthly_attendance`, `v_venue_event_overlap`, `v_voucher_availability`; functions `event_is_verified()`, `events_nearby()`, `find_overlapping_events()`, `haversine_km()`, `recommend_events()`, `search_events()`.

6 Java data-access layer

Entities

28 entity classes in `com.ayojon.entity`, one per table. Composite-key tables use `@EmbeddedId` with `@MapsId`. Columns the database fills in (timestamps, points, published and resolved times) are read-only and re-read after saving. `Event.isVerified()` is computed from the latest verification.

Repositories

28 Spring Data repositories in `com.ayojon.repository`. Native queries over the views return typed projections such as `EventCard`, `LeaderboardEntry` and `ItinerarySlot`. Features call methods like `eventRepository.search(…)` instead of writing SQL.

Security on Supabase

Supabase publishes every table in the `public` schema through a REST API that anyone with the project's public key can call. Without protection that would expose `users.password_hash`. Migration V5 enables Row Level Security on every table and revokes the API roles' access. The Spring Boot backend connects as the table owner and is unaffected.

Demo data

16 events at 12 real Dhaka venues (Hatirjheel Amphitheatre, TSC, Rabindra Sarobar, Bengal Shilpalaya, ICCB, Lalbagh Fort and others), with every status represented. Dates are relative to the load time, so there are always past events (check-ins, reviews), events this coming weekend (itinerary) and upcoming events (search, trending). Accounts: 1 admin, 3 organizers and 5 users; password `password123`, stored as a BCrypt hash.

7 Verification

Everything was tested locally on PostgreSQL 17.5, starting from an empty database.

CHECK	RESULT
App start on an empty database: Flyway applies V1–V5 and the demo data	6 migrations applied
Hibernate `ddl-auto=validate`: every entity matches its table	passed
`verify_business_rules.sql`: valid data accepted, invalid data rejected, across all features	49 / 49 checks passed

CHECK	RESULT
`./mvnw test` : `DatabaseIntegrationTest` runs every custom query plus submit → approve → check-in	**6 / 6 tests passed**
`drop_legacy_draft_tables.sql` on a draft database, and on a migrated one	**drops 24 draft tables / refuses safely**

Examples of rules proven by the SQL checks: a plain user cannot create an event; a check-in at an unverified event earns 0 points while a verified one awards its points and grants a badge; a review without attendance is rejected; a voucher cannot be claimed beyond its stock or without enough points; rescheduling notifies followers and writes the audit log.

How the team tests and merges

```
git fetch origin
git checkout feature/tasnia-database
# update application.properties from the example (ddl-auto=validate + Flyway lines)
./mvnw spring-boot:run      # expect "Successfully applied 6 migrations" on the first run
./mvnw test                 # expect 6 tests, 0 failures
```

If it works, open the pull request from `feature/tasnia-database` into `main`, review and merge.

8 ER diagram

Generated from the migrated database. Crow's-foot notation, lines attach to the key columns.

